# 🏆 AWS CDS Agentic AI Partner Hackathon — Complete Submission Kit

> **Project Name:** Disney Crowd Sense AI
> **Tagline:** Real-time queue headcount vision with Amazon Rekognition, dynamic crowd routing with Amazon Bedrock Nova Lite, and two-way WhatsApp concierge via AWS End User Messaging Social.
> **Marketing Campaign Code:** `AWS CDS Agentic AI Hackathon -Sept. 2026`

## 📋 Table of Contents


1. Devpost Submission Form Fields (Copy & Paste Ready)
2. Architecture Diagram
3. ~3-Minute Video Demo Script & Screen Recording Storyboard
4. Judge & Testing Interaction Instructions
5. AWS Partner Central (ACE) Opportunity Creation Guide
6. Pre-Submission Checklist


## 1. Devpost Submission Form Fields

### 📌 Project Title

```
text
Disney Crowd Sense AI: Real-Time Queue Vision & Two-Way WhatsApp Concierge
```

### 📌 Elevator Pitch / Tagline (100 characters max)

```
text
Zero-app theme park queue optimization powered by AWS CDS WhatsApp, Rekognition & Bedrock.
```

## 📌 About the Project / Description

### 💡 Inspiration

Theme park visitors spend up to **60% of their day waiting in lines**, causing severe vacation fatigue, missed experiences, and frustration. Existing park mobile apps require large app downloads, drain phone batteries, and force visitors to manually refresh static posted times. Furthermore, guests crowd into peak queues unaware that an identical or complementary attraction nearby has a 10-minute walk-on line.

We asked ourselves: *What if visitors didn't need to download an app at all? What if they could communicate with an intelligent Disney AI Concierge right inside WhatsApp—the app they already use—backed by computer vision cameras counting real-time queue density?*

### 🚀 What it does

**Disney Crowd Sense AI** is an intelligent, zero-download park concierge that combines camera-based computer vision with real-time conversational messaging:

1. **Live Camera Queue Headcount Estimation**: Utilizes **Amazon Rekognition** (`DetectLabels`) to analyze attraction camera snapshots, counting actual guests in queue and recomputing wait times without manual sensor maintenance.
2. **Natural Language WhatsApp Concierge**: Powered by **AWS End User Messaging Social** (WhatsApp WABA) and **Amazon Bedrock Nova Lite**, visitors chat with the Disney Assistant naturally on WhatsApp (e.g. "*Can I go Indiana Jones Adventure now?*" or "*Which rides have the shortest wait right now?*").
3. **Proactive Crowd Diversion**: When a guest asks about a congested attraction (e.g. Space Mountain at 75 mins), the AI automatically suggests nearby alternatives with low wait times (e.g. Star Tours at 15 mins).
4. **Queue Drop Alerts & Snapshot Delivery**: Guests can set wait-drop alerts. When the queue clears up, the system automatically pushes an alert—including the analyzed camera snapshot stored in **Amazon S3**—directly to the guest's WhatsApp!

### 🛠️ How we built it

- **AWS End User Messaging Social (CDS Core)**: Linked a Meta WhatsApp Business Account (`waba-3bcb1c396ec446dc8944a2639a397264`) with dedicated bot origination phone `+918072235899`. Handles inbound guest webhooks and dispatches outbound conversational replies and media alerts.
- **Amazon SNS & SQS**: Inbound WhatsApp events route through an SNS topic (`disney-whatsapp-inbound-topic`) to an SQS queue (`disney-whatsapp-inbound-queue`). An event-driven background poller achieves **$0 idle cost** with instant reactive wakeup.

- **Amazon Rekognition**: Processes ingested camera snapshots (`DetectLabels`), identifying human instances in queues to compute dynamic wait times.
- **Amazon Bedrock (Nova Lite v1)**: Generates fast, multi-turn, Disney-themed concierge responses grounded in live park metrics.
- **Amazon DynamoDB**: On-Demand single-digit millisecond NoSQL tables (`Rides` and `UserAlerts`) storing attraction queue headcounts and guest alert thresholds.
- **Amazon S3**: High-durability storage for camera snapshot frames (`crowd-sense-snapshots-1792393252667`) with pre-signed URLs delivered directly over WhatsApp.
- **Admin Operations Dashboard**: Built with Node.js Express, Tailwind CSS, and Server-Sent Events (SSE) to monitor real-time WhatsApp dispatches and trigger camera ingestion per ride.

### 🚧 Challenges we ran into

- **Escaped Meta Webhook JSON Envelopes**: AWS End User Messaging Social wraps Meta's webhook entries inside an escaped JSON string (`whatsAppWebhookEntry`) within an outer context. We built an adaptive parser that seamlessly handles both Meta's raw schema and AWS Social Messaging event envelopes.
- **Zero Idle Cost Constraint**: We avoided expensive always-on resources by designing the architecture around SQS long-polling, DynamoDB on-demand billing, and serverless S3/Bedrock invocations, achieving continuous readiness at virtually zero idle cost.
- **Per-Attraction Snapshot Ingestion**: Instead of a generic upload, we designed a targeted ingestion pipeline allowing park operators to upload camera frames for *any* attraction and immediately dispatch the photo and analysis to real-time WhatsApp.

### 🏅 Accomplishments that we're proud of

- **100% Live End-to-End Execution**: Real two-way WhatsApp messaging verified with live delivery receipts directly from Meta on real mobile devices.
- **Multimodal Real-Time Delivery**: Delivering not just text, but live Rekognition camera snapshot photos directly to guests on WhatsApp via pre-signed S3 URLs.
- **Zero App Download Barrier**: Eliminates the need for any proprietary app download; guests interact via their native WhatsApp client.

### 📚 What we learned

- The power and agility of **AWS End User Messaging Social** for conversational enterprise messaging.
- How combining **Amazon Rekognition** vision with **Amazon Bedrock** language models creates an autonomous "see and advise" agentic loop.

- Best practices for SQS long-polling and SNS event fan-out in AWS Communication Developer Services.

### 🔮 What's next for Disney Crowd Sense AI

- Integration with park ticketing & Lightning Lane / FastPass reservation APIs.
- Multi-lingual voice messaging support using AWS Transcribe and Polly over WhatsApp audio notes.
- Automated drone and stationary CCTV camera streaming ingestion via Amazon Kinesis Video Streams.

### 📌 Built With (Tags for Devpost)

`aws-end-user-messaging-social`, `amazon-bedrock`, `amazon-rekognition`, `amazon-dynamodb`, `amazon-s3`, `amazon-sqs`, `amazon-sns`, `nodejs`, `express`, `tailwind-css`, `whatsapp-business-api`

## 2. Architecture Diagram

## 3. ~3-Minute Video Demo Script & Storyboard

| Timestamp | Visual on Screen | Spoken Script / Audio |
|---|---|---|
| **0:00 - 0:35** | Title slide & problem visuals (long theme park lines). | *"Theme park guests spend up to 60% of their park day waiting in lines. Current theme park apps force visitors to download bulky apps, drain their batteries, and manually refresh static posted times. We built **Disney Crowd Sense AI**—a zero-friction, intelligent park concierge that lives right inside WhatsApp, powered by AWS Communication Developer Services, Amazon Rekognition, and Amazon Bedrock."* |
| **0:35 - 1:10** | Show Architecture Diagram highlighting AWS CDS (WABA), Bedrock, Rekognition, DynamoDB, SQS. | *"Our solution utilizes **AWS End User Messaging Social** connected to a Meta WhatsApp Business Account. Camera snapshots are ingested into Amazon S3 and analyzed by **Amazon Rekognition** to detect real queue headcounts. Wait times update in **Amazon DynamoDB**, while **Amazon Bedrock Nova Lite** handles natural language dialogue and proactive crowd routing."* |
| **1:10 - 1:55** | **Live WhatsApp Demo**: Show real phone screen chatting with `+918072235899`. Send: "*Can I go Indiana Jones Adventure now?*" | *"Let's see it live. A guest opens WhatsApp and asks our verified Disney bot: 'Can I go Indiana Jones Adventure now?'. Through AWS End User Messaging Social, the webhook hits Amazon SNS and SQS. Bedrock Nova Lite checks live DynamoDB metrics and replies instantly: 'Indiana Jones is currently at 35 mins—steady and great to visit!'"* |
| **1:55 - 2:35** | **Live Rekognition & Photo Delivery**: On dashboard at `http://localhost:5000`, click **"📸 Upload Feed"** for Indiana Jones. Upload image. Show WhatsApp on phone receiving the photo. | *"Now, park cameras capture a new queue snapshot. The operator uploads the frame. Amazon Rekognition detects the headcount, updates DynamoDB, and immediately dispatches the camera photo with estimated wait times directly to the guest's WhatsApp via AWS Social Messaging with an S3 pre-signed link!"* |
| **2:35 - 3:00** | Show Operations Dashboard live activity stream & summary slide. | *"In summary, Disney Crowd Sense AI eliminates app fatigue, distributes park crowds intelligently, and runs at virtually $0 idle cost on AWS serverless infrastructure. Thank you!"* |

## 4. Judge & Testing Interaction Instructions

### How Judges Can Test the Live WhatsApp Concierge Right Now:

1. Open **WhatsApp** on your mobile phone.

2. Message the verified Disney AI Concierge at: `+91 80722 35899`
3. Test with any of the following natural language questions:
    - "*Can I go Indiana Jones Adventure now?*"
    - "*Which rides have the shortest wait right now?*"
    - "*What is the wait time for Space Mountain?*"
    - "*Alert me when Space Mountain drops under 30 mins*"
4. The bot will respond in real time powered by live AWS Bedrock and DynamoDB.

### How to Run the Project Locally:

1. Clone the repository and navigate to the backend:

```
bash
git clone <REPO_URL>
cd crowd-sense-ai/backend-node
```

2. Install dependencies:

```
bash
npm install
```

3. Copy environment configuration:

```
bash
cp .env.example .env
```

4. Start the server:

```
bash
npm start
```

5. Open your browser at `http://localhost:5000` to access the live Park Attraction Feeds, Rekognition Vision Analyzer, and Real-Time WhatsApp Operations Center.

## 5. AWS Partner Central (ACE) Opportunity Guide

> **IMPORTANT**

> The hackathon submission requires an **Opportunity ID** created through AWS Partner Central's AWS Customer Engagement (ACE) containing the Marketing Campaign code `AWS CDS Agentic AI Hackathon -Sept. 2026`.

## Step-by-Step Instructions to Create the Opportunity in ACE:

1. Log in to AWS Partner Central.
2. In the navigation menu, select **ACE (AWS Customer Engagement)** → → **Opportunities**.
3. Click **Create Opportunity** (or **New Opportunity**).
4. Fill in the required fields:
    - **Customer Company Name**: Your organization or target enterprise customer (e.g., *Disney Theme Parks & Resorts / Entertainment Demo*).
    - **Project / Opportunity Name**: `Disney Crowd Sense AI - AWS CDS WhatsApp Concierge`
    - **Project Description**: Include a brief 2-sentence summary:
        - *"AI-powered theme park crowd navigation and queue vision analysis leveraging AWS End User Messaging Social (WhatsApp), Amazon Rekognition, and Amazon Bedrock Nova Lite."*
    - **Expected Close Date**: Choose an upcoming date (e.g., end of Q4 2026).
    - **Primary AWS Services**: Select:
        - `AWS End User Messaging` (Communication Developer Services)
        - `Amazon Bedrock`
        - `Amazon Rekognition`
        - `Amazon DynamoDB`
5. **Campaign Code (CRITICAL)**:
    - Find the field labeled **Campaign / Marketing Code** or **Campaign Name**.
    - Enter the exact string:

      ```
      text
      AWS CDS Agentic AI Hackathon -Sept. 2026
      ```
6. Click **Save** / **Submit**.
7. Copy the generated **Opportunity ID** (e.g., `OPP-XXXXXXX`) and paste it into the Devpost submission field!

## 6. Pre-Submission Checklist

- ✓ **Open-Source License**: `LICENSE` (MIT) created and present in the repository root.
- ✓ **Repository Documentation**: `README.md` updated with architecture diagram, AWS CDS deep-dive, and setup guide.
- ✓ **AWS CDS Runtime Demonstration**: Tested and verified live using `@aws-sdk/client-socialmessaging` on WABA `waba-3bcb1c396ec446dc8944a2639a397264`.
- ✓ **Devpost Text Description**: Ready in Section 1 above.
- ✓ **Architecture Diagram**: Mermaid diagram ready in Section 2 above.
- ✓ **3-Minute Video Script**: Storyboard and cues ready in Section 3 above.
- **Push to GitHub / GitLab**: Push the codebase to your public or private repository. (If private, invite `testing@devpost.com` and `aws-cds-partner@amazon.com`).
- **Record Demo Video**: Record the ~3-minute video using the provided script and upload to YouTube/Vimeo.
- **Create ACE Opportunity ID**: Complete the step in Section 5 with code `AWS CDS Agentic AI Hackathon -Sept. 2026`.